

Intermediate RAG

Advanced chunking, hybrid retrieval, re-ranking, query transformation,
multi-vector indexing

01 Advanced Chunking Strategies

Semantic Chunking — Step by Step

Standard fixed-size chunking ignores topic boundaries. Semantic chunking places chunk boundaries where the topic actually changes, detected by a drop in cosine similarity between adjacent sentences.

Step	Action	Detail
1	Split document into sentences	Use NLTK sent_tokenize or spaCy sentence detector
2	Embed every sentence individually	Use a fast embedding model like all-MiniLM-L6-v2
3	Compute cosine sim between adjacent pairs	$\text{sim}(\text{sent}_i, \text{sent}_{i+1})$ for every i
4	Detect breakpoints	Where sim drops below threshold (e.g. $\text{mean} - 1.5 \times \text{std}$)
5	Group sentences between breakpoints	Each group = one semantically coherent chunk

```
# Semantic chunking implementation
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings()

# percentile: split where similarity drops in bottom X%
splitter = SemanticChunker(
    embeddings,
    breakpoint_threshold_type="percentile",
    breakpoint_threshold_amount=95 # more splits = smaller chunks
)

chunks = splitter.split_text(long_document)
# Each chunk contains one coherent topic
```

Parent-Document Retrieval — Precision + Context

The best of both worlds: embed small child chunks for precise matching, but return their larger parent chunks to the LLM for rich context. A child chunk is ~128 tokens; its parent is ~512 tokens.

```
# Parent-Document Retriever (LangChain)
from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

# Two splitters: small for indexing, large for storing
child_splitter = RecursiveCharacterTextSplitter(chunk_size=200)
parent_splitter = RecursiveCharacterTextSplitter(chunk_size=1000)

vectorstore = Chroma(embedding_function=OpenAIEmbeddings())
docstore = InMemoryStore() # stores parent chunks

retriever = ParentDocumentRetriever(
    vectorstore=vectorstore,
    docstore=docstore,
    child_splitter=child_splitter,
    parent_splitter=parent_splitter,
)

retriever.add_documents(docs)

# Query: matches small child chunks, returns large parents
results = retriever.invoke("What is the refund policy?")
```

Hierarchical (RAPTOR) Indexing

RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval) builds a tree: raw chunks at leaves, LLM-generated summaries at intermediate nodes, and a global summary at root. Queries can retrieve at any level — specific facts from leaves or broad context from summaries.

```
# RAPTOR tree building concept
# Step 1: Create leaf chunks (normal chunking)
leaf_chunks = splitter.split_documents(docs)

# Step 2: Cluster leaves into groups (k-means on embeddings)
from sklearn.cluster import KMeans
embeddings_matrix = embed_model.encode([c.page_content for c in leaf_chunks])
)
kmeans = KMeans(n_clusters=10)
labels = kmeans.fit_predict(embeddings_matrix)

# Step 3: LLM summarizes each cluster → intermediate node
for cluster_id in range(10):
    cluster_texts = [leaf_chunks[i].page_content
```

```

        for i,l in enumerate(labels) if l==cluster_id]
    summary = llm.invoke(f"Summarize: {chr(10).join(cluster_texts)}")
    intermediate_nodes.append(summary)

# Step 4: Index ALL levels (leaves + summaries) in vector store
# At query time: retrieve from any level

```

02 Query Transformation Techniques

Multi-Query Retrieval

A single query may miss relevant chunks due to vocabulary mismatch. Generate multiple alternative phrasings of the query, retrieve for each, and merge (deduplicate) results. Significantly improves recall.

```

# Multi-query retriever
from langchain.retrievers.multi_query import MultiQueryRetriever
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(temperature=0)
retriever = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(search_kwargs={"k": 5}),
    llm=llm
)

# LLM generates 3 variations of your query automatically
# Results from all 3 are merged and deduplicated
docs = retriever.invoke("What are the return conditions?")

```

HyDE — Hypothetical Document Embeddings

Ask the LLM to write a hypothetical answer to the query. Embed the hypothetical answer (not the query) and use that vector for retrieval. The generated text is much closer in embedding space to real answer documents than the raw question is.

```

# HyDE implementation
from langchain.chains import HypotheticalDocumentEmbedder
from langchain_openai import OpenAIEmbeddings, ChatOpenAI

base_embeddings = OpenAIEmbeddings()
llm = ChatOpenAI()

embeddings = HypotheticalDocumentEmbedder.from_llm(
    llm=llm,
    base_embeddings=base_embeddings,
    custom_prompt=("Write a document that would answer this: {question}")
)

# Now use these embeddings in your vector store
vectorstore = Chroma(embedding_function=embeddings)

# Query will auto-generate a hypothesis and embed that instead

```

Step-Back Prompting

When a specific factual query fails, abstract it to a more general question. Retrieve for the general question, then answer the specific one using the broader context retrieved.

```
# Step-back prompting
step_back_prompt = """
Given this specific question, generate a more general
"step-back" question that covers the underlying concept:
Specific: {question}
General: """

specific_q = "What is the return policy for electronics purchased in December?"
general_q = llm.invoke(step_back_prompt.format(question=specific_q))
# → "What is the general return policy and any seasonal exceptions?"

# Retrieve for BOTH questions, merge context
context_specific = retriever.invoke(specific_q)
context_general = retriever.invoke(general_q)
combined_context = context_specific + context_general
```

03 Hybrid Search — Deep Dive

BM25 + Dense with RRF — Full Implementation

```
# Hybrid search with BM25 + Qdrant
from langchain_community.retrievers import BM25Retriever
from langchain.retrievers import EnsembleRetriever

# BM25 (sparse) retriever
bm25_retriever = BM25Retriever.from_documents(chunks)
bm25_retriever.k = 10

# Dense (vector) retriever
dense_retriever = vectorstore.as_retriever(search_kwargs={"k": 10})

# Ensemble with RRF fusion (weights control influence)
hybrid_retriever = EnsembleRetriever(
    retrievers=[bm25_retriever, dense_retriever],
    weights=[0.4, 0.6] # 40% sparse, 60% dense
)

docs = hybrid_retriever.invoke("Python async error handling")
# Returns merged, deduplicated, RRF-ranked results
```

Cross-Encoder Re-ranking — The Accuracy Boost

Bi-encoders (used in retrieval) encode query and document independently — fast but less accurate. Cross-encoders read query+document together — much more accurate but too slow for full-corpus

search. Solution: retrieve top-50 cheaply, re-rank with cross-encoder, pass top-5 to LLM.

```
# Re-ranking with Cohere or local cross-encoder
# Option A: Cohere Rerank API
import cohere
co = cohere.Client("your-api-key")

initial_docs = retriever.invoke(query) # get top-20
reranked = co.rerank(
    query=query,
    documents=[d.page_content for d in initial_docs],
    top_n=5,
    model="rerank-english-v3.0"
)

# Option B: Local cross-encoder (free)
from sentence_transformers import CrossEncoder
reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

pairs = [(query, doc.page_content) for doc in initial_docs]
scores = reranker.predict(pairs)
ranked = sorted(zip(scores, initial_docs), reverse=True)
top_5 = [doc for _, doc in ranked[:5]]
```

04 Advanced RAG Architecture Patterns

Corrective RAG (CRAG)

After retrieving, evaluate whether the retrieved documents are actually relevant. If not — or if confidence is low — trigger a web search to supplement or replace the context before generation.

```
# CRAG decision logic
def evaluate_relevance(query, docs, llm):
    prompt = f"""Rate relevance of these docs to the query.
    Query: {query}
    Docs: {[d.page_content[:200] for d in docs]}
    Score: RELEVANT / AMBIGUOUS / IRRELEVANT"""
    return llm.invoke(prompt)

score = evaluate_relevance(query, retrieved_docs, llm)

if score == "IRRELEVANT":
    # Fall back to web search
    web_docs = web_search_tool.invoke(query)
    context = web_docs
elif score == "AMBIGUOUS":
    # Combine local + web
    web_docs = web_search_tool.invoke(query)
```

```
context = retrieved_docs + web_docs
else:
    context = retrieved_docs
```

Self-RAG — Retrieve Only When Needed

A fine-tuned model that decides: (1) whether to retrieve at all, (2) whether retrieved docs are relevant, (3) whether the answer is grounded, (4) whether the answer is useful. Adds special tokens to control the generation process. More accurate but requires a special model.

Agentic RAG — RAG Inside an Agent Loop

Instead of one retrieval step, an agent iteratively retrieves — reading the initial results, identifying gaps, forming follow-up queries, retrieving again, and repeating until confident it has enough context to answer well.

```
# Agentic RAG with LangGraph
from langgraph.graph import StateGraph, END
from typing import TypedDict, List

class RAGState(TypedDict):
    query: str
    retrieved_docs: List
    answer: str
    iterations: int

def retrieve(state):
    docs = retriever.invoke(state["query"])
    return {**state, "retrieved_docs": docs}

def generate(state):
    answer = rag_chain.invoke(state)
    return {**state, "answer": answer}

def check_answer(state):
    # LLM evaluates if answer is complete
    if "I need more information" in state["answer"]:
        return "retrieve" # loop back
    return END

graph = StateGraph(RAGState)
graph.add_node("retrieve", retrieve)
graph.add_node("generate", generate)
graph.add_edge("retrieve", "generate")
graph.add_conditional_edges("generate", check_answer)
graph.set_entry_point("retrieve")
app = graph.compile()
```

Custom RAGAS Metrics

```
# Custom metric: Response Completeness
from ragas.metrics.base import MetricWithLLM, SingleTurnMetric
from ragas.prompt import PydanticPrompt

class CompletenessScore(MetricWithLLM, SingleTurnMetric):
    name = "completeness"

    async def _single_turn_ascore(self, sample, callbacks):
        prompt = f"""Does this answer completely address all parts
        of the question?
        Question: {sample.user_input}
        Answer: {sample.response}
        Score 0-1 (1=fully complete):"""
        result = await self.llm.generate(prompt)
        return float(result.generations[0][0].text.strip())
```

Automated Evaluation Pipeline

```
# Full eval pipeline with test set generation + scoring
import pandas as pd
from ragas.testset import TestsetGenerator
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy,
                        context_precision, context_recall

# Step 1: Generate test set from your documents
generator = TestsetGenerator.from_langchain(llm=llm, embeddings=emb)
testset = generator.generate_with_langchain_docs(
    documents=docs, test_size=50,
    distributions={"simple": 0.5, "multi_context": 0.3,
                  "reasoning": 0.2})

# Step 2: Run your RAG on each test question
answers, contexts = [], []
for q in testset.to_pandas()["question"]:
    result = rag_chain.invoke({"query": q})
    answers.append(result["result"])
    contexts.append([d.page_content for d in result["source_documents"]])

# Step 3: Evaluate
eval_data = testset.to_pandas().copy()
```

```

eval_data["answer"] = answers
eval_data["contexts"] = contexts

scores = evaluate(Dataset.from_pandas(eval_data),
                  metrics=[faithfulness, answer_relevancy,
                           context_precision, context_recall])

print(scores)

```

A/B Testing Retrieval Strategies

```

# Compare two retrieval strategies on same test set
def eval_strategy(retriever, test_df, metrics):
    results = []
    for _, row in test_df.iterrows():
        docs = retriever.invoke(row["question"])
        answer = generate(row["question"], docs)
        results.append({"question": row["question"],
                       "answer": answer,
                       "contexts": [d.page_content for d in docs],
                       "ground_truth": row["ground_truth"]})
    return evaluate(Dataset.from_list(results), metrics=metrics)

# Compare
score_dense = eval_strategy(dense_retriever, test_df, metrics)
score_hybrid = eval_strategy(hybrid_retriever, test_df, metrics)
score_rerank = eval_strategy(rerank_retriever, test_df, metrics)

comparison = pd.DataFrame({
    "Dense Only": score_dense,
    "Hybrid": score_hybrid,
    "Hybrid+Rerank": score_rerank
})

print(comparison) # See which strategy wins on your data

```